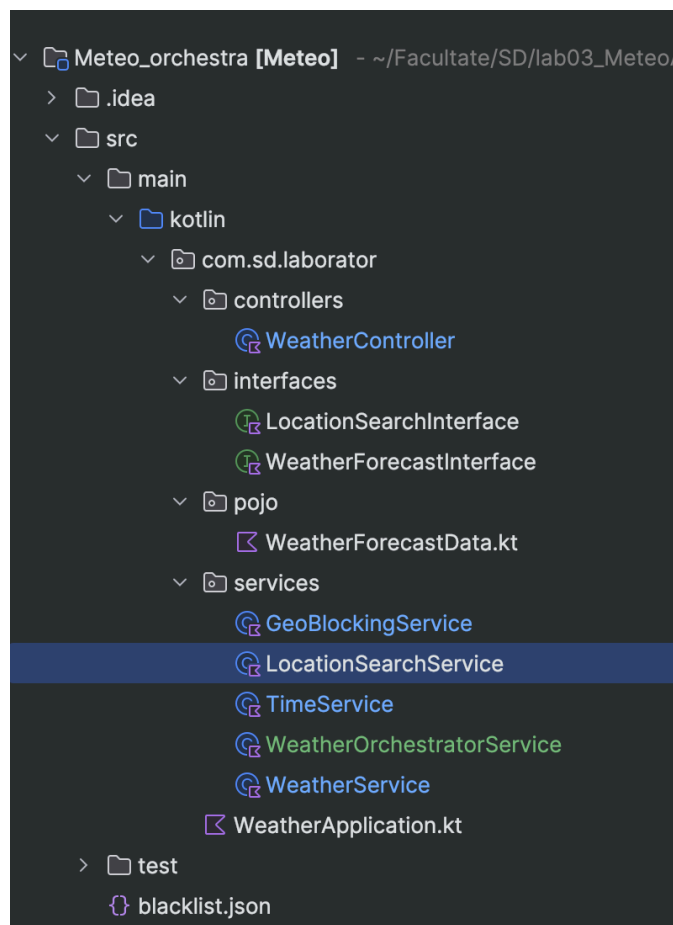


Pornind de la exemplul de proiectare si implementare a unei aplicatii care primeste informatii meteo (adică utilizând si serviciile de acolo eventual cu modificări) să se realizeze următoarele:

- * un serviciu care citește dintr-un fisier (in ce format dorim) o listă de orase;
- * să se aplice modelul de proiectare de tip filtru în proiectarea si implementarea unui serviciu care extrage din fluxul de date informatiil pe baza numelui orasului (utilizând serviciul anterior)
- * să se proiecteze si să se implementeze un serviciu care utilizând serviciul de filtrare va aplica o regresie liniară peste datele rezultate si apoi se va afisa unghiul dreptei rezultate.

Daca se considera necesar se mai pot crea oricâte servicii se doreste conform gândirii studentului.

Se va utiliza orchestrarea si respectarea principillor SOLID pentru microservicii. La prezentare se va arata diagrama de clase si se va explica aceasta impreuna cu gradul de respectare a principillor. Apoi va urma discutia pe cod.



NOTITE

1. Stai calma am sters git-ul :)

WeatherController:

```
package com.sd.laborator.controllers

import com.sd.laborator.services.WeatherOrchestratorService
import org.springframework.web.bind.annotation.*
import reactor.core.publisher.Mono

@RestController
@RequestMapping("/weather")
class WeatherController(private val weatherOrchestratorService:
WeatherOrchestratorService) {

    @GetMapping("/current")
    fun getCurrentWeather(
        @RequestParam latitude: Double,
        @RequestParam longitude: Double
    ): Mono<Any> {
        return weatherOrchestratorService.getWeather(latitude, longitude)
    }
}
```

LocationSearchInterface

```
package com.sd.laborator.interfaces

interface LocationSearchInterface {
    fun getLocationId(locationName: String): Int
}
```

WeatherForecastInterface

```
package com.sd.laborator.interfaces

import com.sd.laborator.pojo.WeatherForecastData

interface WeatherForecastInterface {
    fun getForecastData(locationId: Int): WeatherForecastData
}
```

WeatherForecastData

```
package com.sd.laborator.pojo

data class WeatherForecastData(
```

```

        val latitude: Double,
        val longitude: Double,
        val generationtime_ms: Double,
        val utc_offset_seconds: Int,
        val timezone: String,
        val timezone_abbreviation: String,
        val elevation: Double,
        val hourly_units: HourlyUnits,
        val hourly: HourlyData
    )

    data class HourlyUnits(
        val time: String,
        val temperature_2m: String
    )

    data class HourlyData(
        val time: List<String>,
        val temperature_2m: List<Double>
    )

```

GeoBlockingService

```

package com.sd.laborator.services

import org.springframework.stereotype.Service
import java.io.File
import java.net.URL
import com.fasterxml.jackson.module.kotlin.jacksonObjectMapper
import com.fasterxml.jackson.module.kotlin.readValue
import reactor.core.publisher.Mono

@Service
class GeoBlockingService {
    private val objectMapper = jacksonObjectMapper()

    fun getLocalGeoLocationMono(): Mono<String> {
        return Mono.fromCallable {
            try {
                val response = URL("https://ipapi.co/json/").readText()
                val locationData: Map<String, Any> =
                    objectMapper.readValue(response)
                locationData["country_name"] as? String ?: "Unknown"
            } catch (e: Exception) {
                "Unknown"
            }
        }
    }
}

```

```

    fun isBlockedLocationMono(location: String): Mono<Boolean> {
        return Mono.fromCallable {
            val blacklistFile =
File("/Users/andreighiurca/Facultate/SD/lab03_MeteoApp/Meteo_orchestra/src/bla
cklist.json")
            if (!blacklistFile.exists()) return@fromCallable false

            val blacklistedCountries: List<String> =
objectMapper.readValue(blacklistFile)
            blacklistedCountries.contains(location)
        }
    }
}

```

LocationSearchService

```

package com.sd.laborator.services

import com.sd.laborator.interfaces.LocationSearchInterface
import org.springframework.stereotype.Service
import java.net.URL
import org.json.JSONObject
import java.net.URLEncoder
import java.nio.charset.StandardCharsets
@Service
class LocationSearchService : LocationSearchInterface {
    override fun getLocationId(locationName: String): Int {
// codificare parametru URL (deoarece poate conține caracterespeciale)
        val encodedLocationName =
URLEncoder.encode(locationName, StandardCharsets.UTF_8.toString())
// construire obiect de tip URL
        val locationSearchURL =

URL("https://www.metaweather.com/api/location/search/?query=$encodedLocationNa
me")
// preluare raspuns HTTP (se face cerere GET și se preia conținutul
răspunsului sub formă de text)
        val rawResponse: String = locationSearchURL.readText()
// parsare obiect JSON
        val responseObject = JSONObject("{\"data\": ${rawResponse}}")
        val responseContentObject =
            responseObject.getJSONArray("data").takeUnless {
it.isEmpty}??.getJSONObject(0)
        return responseContentObject?.getInt("woeid") ?: -1
    }
}

```

TimeService

```
package com.sd.laborator.services
import org.springframework.stereotype.Service
import java.text.SimpleDateFormat
import java.util.*

@Service
class TimeService {
    fun getCurrentTime():String {
        val formatter = SimpleDateFormat("HH:mm:ss")
        return formatter.format(Date())
    }
}
```

WeatherOrchestratorService

```
package com.sd.laborator.services

import com.sd.laborator.pojo.WeatherForecastData
import org.springframework.stereotype.Service
import reactor.core.publisher.Mono

@Service
class WeatherOrchestratorService(
    private val geoBlockingService: GeoBlockingService,
    private val weatherService: WeatherService
) {
    fun getWeather(latitude: Double, longitude: Double): Mono<Any> {
        return geoBlockingService.getLocalGeoLocationMono()
            .flatMap { location ->
                geoBlockingService.isBlockedLocationMono(location)
                    .flatMap { isBlocked ->
                        if (isBlocked) {
                            Mono.just(
                                mapOf(
                                    "longitude" to longitude,
                                    "latitude" to latitude,
                                    "error" to "Access to weather data is
restricted for location: $location."
                                )
                            )
                        } else {
                            weatherService.getWeatherData(latitude, longitude)
                        }
                    }
            }
    }
}
```

```

    }
}
///Ce e ala mono
//Lanutul trebuie verificat cum merge
//functiile la mono ce fac
//verificat blocarea tarilor ca google imi face o schema

```

WeatherService

```

package com.sd.laborator.services

import com.sd.laborator.pojo.WeatherForecastData
import org.springframework.stereotype.Service
import org.springframework.web.reactive.function.client.WebClient
import reactor.core.publisher.Mono
import java.time.LocalDateTime
import java.time.format.DateTimeFormatter

@Service
class WeatherService(private val geoBlockingService: GeoBlockingService) {
    private val webClient = WebClient.create("https://api.open-meteo.com")

    fun getWeatherData(latitude: Double, longitude: Double): Mono<Any> {
        return geoBlockingService.getLocalGeoLocationMono()
            .flatMap { location ->
                geoBlockingService.isBlockedLocationMono(location)
                    .flatMap { isBlocked ->
                        if (isBlocked) {
                            Mono.just(
                                mapOf(
                                    "longitude" to longitude,
                                    "latitude" to latitude,
                                    "error" to "Access to weather data is
restricted for location: $location."
                                )
                            )
                        } else {
                            fetchWeatherData(latitude, longitude, location)
                        }
                    }
            }
    }

    private fun fetchWeatherData(latitude: Double, longitude: Double, location:
String): Mono<List<Map<String, Any>>> {
        return webClient.get()

```

```

.uri("/v1/forecast?latitude=$latitude&longitude=$longitude&hourly=temperature_2m")
    .retrieve()
    .bodyToMono(WeatherForecastData::class.java)
    .map { data -> formatWeatherResponse(data, location) }
}

private fun formatWeatherResponse(data: WeatherForecastData, location: String): List<Map<String, Any>> {
    val formattedWeather = mutableList<Map<String, Any>>()
    val timeFormatter = DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm")

    for (i in data.hourly.time.indices) {
        val timestamp = LocalDateTime.parse(data.hourly.time[i])
        val formattedDateTime = timestamp.format(timeFormatter).split(" ")

        formattedWeather.add(
            mapOf(
                "latitude" to data.latitude,
                "longitude" to data.longitude,
                "location" to location,
                "date" to formattedDateTime[0], // yyyy-MM-dd
                "hour" to formattedDateTime[1], // HH:mm
                "temperature" to data.hourly.temperature_2m[i]
            )
        )
    }

    return formattedWeather
}
}

```

WeatherApplication

```

package com.sd.laborator

import org.springframework.boot.autoconfigure.SpringBootApplication
import org.springframework.boot.runApplication

@SpringBootApplication
open class WeatherApplication

fun main(args: Array<String>) {
    runApplication<WeatherApplication>(*args)
}

```

Blacklist.json

```
[  
    "North Korea",  
    "Iran",  
    "Russia"  
]
```

Pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>  
<project xmlns="http://maven.apache.org/POM/4.0.0"  
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0  
        http://maven.apache.org/maven-v4_0_0.xsd">  
  
    <modelVersion>4.0.0</modelVersion>  
  
    <groupId>com.sd.laborator</groupId>  
    <artifactId>Meteo</artifactId>  
    <version>1.0-SNAPSHOT</version>  
    <packaging>jar</packaging>  
  
    <name>com.sd.laborator Meteo</name>  
    <properties>  
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>  
        <kotlin.version>2.1.20-Beta2</kotlin.version>  
        <kotlin.code.style>official</kotlin.code.style>  
        <junit.version>4.13.1</junit.version>  
    </properties>  
  
    <parent>  
        <groupId>org.springframework.boot</groupId>  
        <artifactId>spring-boot-starter-parent</artifactId>  
        <version>2.3.12.RELEASE</version>  
        <relativePath/>  
    </parent>  
    <dependencies>  
        <dependency>  
            <groupId>org.jetbrains.kotlin</groupId>  
            <artifactId>kotlin-stdlib</artifactId>  
            <version>${kotlin.version}</version>  
        </dependency>  
        <dependency>  
            <groupId>org.jetbrains.kotlin</groupId>  
            <artifactId>kotlin-test-junit</artifactId>  
            <version>${kotlin.version}</version>  
            <scope>test</scope>  
        </dependency>  
    </dependencies>
```



```

        <groupId>junit</groupId>
        <artifactId>junit</artifactId>
        <version>${junit.version}</version>
        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
        <groupId>org.jetbrains.kotlin</groupId>
        <artifactId>kotlin-reflect</artifactId>
        <version>${kotlin.version}</version>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-devtools</artifactId>
        <version>${project.parent.version}</version>
    </dependency>
    <dependency>
        <groupId>org.json</groupId>
        <artifactId>json</artifactId>
        <version>20210307</version>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-webflux</artifactId>
    </dependency>

    <dependency>
        <groupId>com.fasterxml.jackson.module</groupId>
        <artifactId>jackson-module-kotlin</artifactId>
        <version>2.13.0</version>
    </dependency>
</dependencies>

<build>
    <sourceDirectory>src/main/kotlin</sourceDirectory>
    <testSourceDirectory>src/test/kotlin</testSourceDirectory>

    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
            <configuration>
<mainClass>com.sd.laborator.WeatherApplicationKt</mainClass>
            </configuration>

```

```
</plugin>
<plugin>
  <groupId>org.jetbrains.kotlin</groupId>
  <artifactId>kotlin-maven-plugin</artifactId>
  <version>${kotlin.version}</version>
  <executions>
    <execution>
      <id>compile</id>
      <phase>compile</phase>
      <goals>
        <goal>compile</goal>
      </goals>
      <configuration>
        <compilerPlugins>
          <plugin>spring</plugin>
        </compilerPlugins>
      </configuration>
    </execution>
    <execution>
      <id>test-compile</id>
      <phase>test-compile</phase>
      <goals>
        <goal>test-compile</goal>
      </goals>
    </execution>
  </executions>
  <dependencies>
    <dependency>
      <groupId>org.jetbrains.kotlin</groupId>
      <artifactId>kotlin-maven-allopen</artifactId>
      <version>${kotlin.version}</version>
    </dependency>
  </dependencies>
</plugin>
</plugins>
</build>

</project>
```